



Université Paris Cité

Master 1 Vision et Machine Intelligente

Test-Time Training auto-supervisé pour la classification d'images

Étude expérimentale de SimCLR et Vision Transformers sous
décalage de distribution

Auteur : Maksym Dolhov & Fallou Diouf

Encadrant : Pr Ayoub Karine Pr Camille Kurtz Pr Laurent Wendling

Année universitaire : 2025–2026

Table des matières

Résumé	3
1 Introduction	4
1.1 Contexte	4
1.2 Problématique	4
1.3 Objectifs du TER	5
1.4 Contributions	5
1.5 Organisation du rapport	5
2 État de l’art	6
2.1 Apprentissage auto-supervisé	6
2.1.1 Principe général	6
2.1.2 SimCLR	6
2.1.3 MoCo	7
2.1.4 BYOL	7
2.1.5 DINO	7
2.2 Vision Transformers	7
2.2.1 Architecture ViT	7
2.2.2 Patch Embedding	7
2.2.3 Self-Attention	7
2.2.4 LayerNorm	8
2.3 Distribution Shift	8
2.3.1 Définition	8
2.3.2 Corruptions et perturbations	8
2.3.3 Robustesse des modèles	8
2.4 Test-Time Training	8
2.4.1 TTT de Sun et al.	8
2.4.2 TTT++	9
2.4.3 Tent	9
2.4.4 ActMAD	9
2.5 Limites des approches existantes	9

3	Méthodologie	10
3.1	Vue globale du pipeline	10
3.2	Pré-entraînement auto-supervisé	11
3.2.1	Architecture SimCLR	11
3.2.2	Vision Transformer ViT-Tiny	12
3.2.3	Augmentations de données	12
3.2.4	Perte NT-Xent	12
3.3	Fine-tuning supervisé	12
3.3.1	Tête de classification	12
3.3.2	Tête auxiliaire de rotation	12
3.3.3	Fonction de perte	13
3.4	Test-Time Training	13
3.4.1	Principe du TTT	14
3.4.2	Adaptation per-batch	14
3.4.3	Adaptation per-image	14
3.4.4	Scope d'adaptation	15
4	Étude expérimentale	16
4.1	Datasets	16
4.1.1	CIFAR-10	16
4.1.2	CIFAR-10-C	16
4.1.3	GPU et environnement	16
4.2	Setup expérimental	17
4.2.1	Hyperparamètres	17
4.2.2	Protocoles d'évaluation	18
4.3	Résultats sans TTT	18
4.4	Résultat avec TTT	19
4.4.1	Comparaison globale	20
4.4.2	Effet des hyperparamètres	20
5	Discussion	21
5.1	Pourquoi le per-batch fonctionne	21
5.2	Pourquoi le per-image échoue	21
5.3	Limites de l'étude	22
6	Conclusion et perspectives	23
6.1	Conclusion	23
6.2	Perspectives	24

Résumé

Le décalage de distribution entre les données d'entraînement et les données de test constitue un problème majeur en vision par ordinateur. Les approches de *Test-Time Training* (TTT) visent à adapter un modèle directement au moment de l'inférence à l'aide d'objectifs auto-supervisés, sans accès aux annotations des données de test.

Dans ce travail, nous étudions l'intégration du TTT dans un pipeline de classification d'images fondé sur l'apprentissage auto-supervisé. Nous proposons un pipeline complet combinant un pré-entraînement SimCLR avec un encodeur ViT-Tiny, un fine-tuning supervisé sur CIFAR-10, puis une phase d'adaptation au test sur CIFAR-10-C à l'aide d'une tâche auxiliaire de prédiction de rotation.

Nos expériences montrent que le TTT en mode *per-batch* améliore systématiquement les performances sous dérive de distribution, avec des gains croissants lorsque la sévérité des corruptions augmente. À l'inverse, le TTT *per-image* dégrade les performances dans notre configuration basée sur ViT et LayerNorm. Les ablations montrent que des adaptations plus agressives amplifient simultanément les gains du mode batch et les dégradations du mode image.

Ces résultats suggèrent que l'efficacité du TTT dépend fortement du régime d'adaptation et de l'architecture de normalisation utilisée.

Mots-clés : Self-Supervised Learning, Test-Time Training, Vision Transformer, Distribution Shift, CIFAR-10-C.

Chapitre 1

Introduction

1.1 Contexte

Les modèles modernes de vision par ordinateur reposent généralement sur de grandes quantités de données annotées afin d’obtenir des performances élevées. Cependant, l’annotation manuelle demeure coûteuse, longue et difficilement scalable. L’apprentissage auto-supervisé (*Self-Supervised Learning*, SSL) constitue une alternative prometteuse permettant d’apprendre des représentations visuelles générales à partir de données non annotées.

Dans un pipeline classique, un modèle est d’abord pré-entraîné de manière auto-supervisée sur un large ensemble d’images, puis spécialisé via un fine-tuning supervisé pour une tâche de classification. Toutefois, les distributions des données d’entraînement et de test peuvent différer significativement. Ce phénomène, appelé *distribution shift*, entraîne une dégradation notable des performances lors de l’inférence, en particulier sous bruit, flou, variations d’illumination ou transformations géométriques.

1.2 Problématique

Les modèles de classification d’images sont sensibles aux décalages de distribution. Les données de test peuvent présenter des corruptions ou transformations qui n’étaient pas présentes lors de l’entraînement, ce qui réduit fortement la robustesse du modèle.

Une question centrale se pose alors :

Comment adapter dynamiquement un modèle pendant l’inférence afin d’améliorer sa robustesse, sans utiliser les labels des données de test ?

Le *Test-Time Training* (TTT), introduit par Sun et al. [?], propose une réponse à cette problématique en permettant au modèle de continuer à apprendre pendant l’inférence grâce à une tâche auto-supervisée auxiliaire.

1.3 Objectifs du TER

L’objectif principal de ce TER est d’étudier l’application du Test-Time Training dans un cadre auto-supervisé pour la classification d’images. Plus précisément, nous cherchons à :

- implémenter un modèle auto-supervisé pré-entraîné avec SimCLR ;
- réaliser un fine-tuning supervisé pour la classification d’images ;
- appliquer une adaptation Test-Time Training sur les données de test ;
- comparer deux régimes d’adaptation : *per-batch* et *per-image* ;
- analyser l’impact du décalage de distribution sur CIFAR-10-C.

1.4 Contributions

Dans ce travail, nous proposons :

1. un pipeline complet et reproductible combinant SimCLR, ViT-Tiny, fine-tuning supervisé et TTT basé sur une tâche de rotation ;
2. une comparaison systématique entre les régimes d’adaptation *per-batch* et *per-image* ;
3. une démonstration expérimentale que le TTT *per-batch* améliore systématiquement les performances sous corruption, tandis que le TTT *per-image* dégrade fortement les performances dans notre configuration ViT/LayerNorm ;
4. une analyse de l’effet des hyperparamètres d’adaptation montrant qu’une adaptation plus agressive amplifie simultanément les gains du mode batch et les dégradations du mode image.

1.5 Organisation du rapport

Le reste du mémoire est organisé comme suit :

- le Chapitre 2 présente l’état de l’art sur l’apprentissage auto-supervisé, le distribution shift et le Test-Time Training ;
- le Chapitre 3 décrit la méthodologie proposée, incluant SimCLR, ViT-Tiny, le fine-tuning et l’adaptation TTT ;
- le Chapitre 4 présente les expériences menées sur CIFAR-10 et CIFAR-10-C ;
- le Chapitre 5 analyse les résultats, notamment la comparaison *per-batch* / *per-image* ;
- le Chapitre 6 conclut ce travail et propose des perspectives.

Chapitre 2

État de l’art

2.1 Apprentissage auto-supervisé

2.1.1 Principe général

L’apprentissage auto-supervisé (SSL) vise à apprendre des représentations visuelles sans annotations humaines. Le principe consiste à définir une tâche prétexte (*pretext task*) que le modèle peut résoudre en utilisant uniquement les données brutes. En apprenant à résoudre cette tâche artificielle, le modèle apprend des représentations utiles, transférables et souvent plus robustes que celles issues d’un entraînement supervisé classique.

Les tâches prétextes peuvent être :

- **géométriques** : prédiction de rotation, jigsaw, inpainting ;
- **contextuelles** : ordre de patches, colorisation ;
- **contrastives** : rapprocher deux vues d’une même image.

Les méthodes contrastives ont dominé le domaine depuis 2020.

2.1.2 SimCLR

SimCLR [5] est une méthode contrastive simple et efficace. Deux vues augmentées d’une même image forment une paire positive, tandis que les vues d’images différentes forment des paires négatives. La perte NT-Xent maximise la similarité des positives et minimise celle des négatives.

Les ingrédients clés sont :

- un encodeur (CNN ou ViT) ;
- un projecteur MLP ;
- des augmentations fortes (crop, jitter, blur) ;
- une grande taille de batch.

SimCLR a montré qu’un simple objectif contrastif peut rivaliser avec l’apprentissage supervisé.

2.1.3 MoCo

MoCo [?] introduit un **encodeur à momentum** et une **file d’attente dynamique** de représentations négatives. Cela permet d’utiliser des milliers de négatifs sans augmenter la taille du batch, rendant l’entraînement plus stable et plus efficace.

2.1.4 BYOL

BYOL [?] supprime complètement les négatifs. Il utilise deux réseaux :

- un réseau en ligne ;
- un réseau cible mis à jour par momentum.

Le réseau en ligne apprend à prédire les représentations du réseau cible. Malgré l’absence de négatifs, BYOL évite l’effondrement grâce au momentum encoder.

2.1.5 DINO

DINO [?] applique un schéma enseignant–élève à un Vision Transformer. Il introduit un mécanisme de **centering** et **sharpening** pour stabiliser l’entraînement. DINO a montré que les ViT apprennent spontanément des structures sémantiques (objets, parties) sans supervision.

2.2 Vision Transformers

2.2.1 Architecture ViT

Le Vision Transformer (ViT) [6] applique directement l’architecture Transformer aux images. Une image est découpée en patches, linéarisée, puis traitée comme une séquence de tokens.

2.2.2 Patch Embedding

Chaque patch 16×16 ou 4×4 est projeté dans un espace latent via une couche linéaire. Un token spécial [CLS] est ajouté pour la classification.

2.2.3 Self-Attention

Le mécanisme d’attention permet au modèle de capturer des relations globales entre les patches. Contrairement aux CNN, ViT ne repose pas sur des convolutions locales.

2.2.4 LayerNorm

ViT utilise **LayerNorm** avant chaque bloc Transformer. Cela améliore la stabilité mais rend le modèle plus sensible aux mises à jour locales — un point crucial pour le Test-Time Training.

2.3 Distribution Shift

2.3.1 Définition

Le *distribution shift* désigne une différence entre la distribution des données d'entraînement et celle des données de test. Il entraîne une dégradation des performances, même pour des modèles très performants en conditions propres.

Types de décalages

- **Bruit** : gaussian noise, impulse noise.
- **Flou** : motion blur, defocus blur.
- **Transformations géométriques** : rotation, translation.
- **Changements d'illumination** : brightness, contrast.
- **Domain shift** : changement de style, de capteur, de contexte.

2.3.2 Corruptions et perturbations

Le benchmark CIFAR-10-C [7] propose 15 corruptions réalistes, chacune déclinée en 5 niveaux de sévérité. Il est devenu un standard pour évaluer la robustesse des modèles.

2.3.3 Robustesse des modèles

Les modèles supervisés classiques sont très sensibles aux corruptions. Les modèles SSL sont souvent plus robustes, mais restent vulnérables aux perturbations sévères.

2.4 Test-Time Training

2.4.1 TTT de Sun et al.

Le Test-Time Training (TTT) [1] propose d'adapter le modèle pendant l'inférence grâce à une tâche auto-supervisée. Dans la version originale, la tâche auxiliaire est la **prédiction de rotation**.

Pendant l'entraînement :

- le modèle apprend la classification + la rotation ;

- la tête rotation est conservée pour le test.

Au test :

- les labels ne sont pas disponibles ;
- un pas de gradient est effectué sur la perte rotation ;
- la prédiction finale est produite après adaptation.

2.4.2 TTT++

TTT++ [2] améliore la stabilité du TTT :

- régularisation de l'adaptation ;
- séparation stricte backbone / tête SSL ;
- meilleure gestion des corruptions sévères.

2.4.3 Tent

Tent [?] propose une adaptation plus simple : minimiser l'entropie des prédictions au test, en adaptant uniquement les paramètres de BatchNorm. Tent est efficace mais dépend fortement de la présence de BatchNorm — ce qui n'est pas le cas des ViT.

2.4.4 ActMAD

ActMAD [4] aligne les activations entre données propres et données corrompues. Il s'agit d'une méthode d'adaptation plus stable, mais plus coûteuse.

2.5 Limites des approches existantes

Les méthodes d'adaptation au test présentent plusieurs limites :

- **Coût computationnel** : adaptation par gradient pour chaque batch ou image.
- **Instabilité** : risque de dérive des paramètres, surtout pour les ViT.
- **Sensibilité aux hyperparamètres** : learning rate, nombre de steps, scope d'adaptation.
- **Dépendance à BatchNorm** : Tent et TTT original fonctionnent mieux sur ResNet que sur ViT.
- **Catastrophic forgetting** : adaptation trop agressive $\Rightarrow$ perte de performance sur les données propres.

Chapitre 3

Méthodologie

3.1 Vue globale du pipeline

Le pipeline complet mis en place dans ce travail comporte quatre étapes principales, illustrant la progression du modèle depuis l'apprentissage auto-supervisé jusqu'à l'adaptation au moment du test :

Stage A — Pré-entraînement SimCLR (ViT-Tiny). Le modèle est pré-entraîné de manière auto-supervisée à l'aide de SimCLR. Deux vues augmentées sont générées pour chaque image (crop, jitter, grayscale, blur), puis encodées par un Vision Transformer ViT-Tiny. La perte contrastive NT-Xent est appliquée sur les représentations projetées. Ce pré-entraînement produit un encodeur robuste, sauvegardé sous la forme `encoder_pretrained.pt`.

Stage B.1 — Linear Probe (évaluation des représentations). L'encodeur pré-entraîné est gelé et une couche linéaire est entraînée pendant 30 époques. Ce stage permet d'évaluer la qualité des représentations apprises par SimCLR indépendamment du fine-tuning complet.

Stage B.2 — Fine-tuning supervisé avec tête auxiliaire. L'encodeur est dégelé et deux têtes sont ajoutées :

- une tête de classification (10 classes) optimisée avec une cross-entropy et du label smoothing ;
- une tête auxiliaire de prédiction de rotation (4 classes : 0° , 90° , 180° , 270°).

Les deux pertes sont combinées pendant l'entraînement. La tête rotation est conservée pour l'adaptation TTT.

Stage C — Évaluation CIFAR-10-C et Test-Time Training. Le modèle est évalué sur CIFAR-10-C (14 corruptions $\times$ 5 sévérités). Deux modes d'adaptation sont étudiés :

- **TTT per-batch** : un pas de gradient par batch de test ;
- **TTT per-image** : un pas de gradient par image (batch artificiel de 128 rotations).

Les résultats sont enregistrés dans `cifar10c_results.csv`. Les poids sont réinitialisés entre chaque corruption et sévérité pour éviter toute fuite d'adaptation.

Ce pipeline est entièrement configurable via `configs/default.yaml`, avec gestion des seeds, AMP, AdamW, warmup et scheduler cosine. Deux configurations sont proposées : un mode *smoke* (5 minutes) et un mode *default* (run overnight sur GPU L4).

3.2 Pré-entraînement auto-supervisé

3.2.1 Architecture SimCLR

L'architecture SimCLR utilisée dans ce travail suit fidèlement la formulation originale de Chen et al. (2020), adaptée à un backbone Vision Transformer ViT-Tiny. Le principe consiste à encoder deux vues augmentées d'une même image, puis à projeter leurs représentations dans un espace latent où la perte contrastive NT-Xent est appliquée.

Notre configuration SimCLR est définie dans le fichier `configs/default.yaml` et repose sur les hyperparamètres suivants :

- **epochs : 200** — durée d'entraînement suffisante pour stabiliser les représentations ViT ;
- **temperature : 0.2** — contrôle la concentration de la distribution softmax dans la perte NT-Xent ;
- **learning_rate : 0.003** — taux d'apprentissage initial utilisé avec un scheduler cosine ;
- **weight_decay : 10^{-4}** — régularisation L2 appliquée à l'encodeur ;
- **warmup_epochs : 10** — montée progressive du learning rate pour éviter les instabilités initiales ;
- **use_amp : true** — entraînement en précision mixte (AMP) pour réduire le coût mémoire ;
- **log_every_n_steps : 20** — fréquence de journalisation des métriques.

L'architecture complète se compose de trois modules :

1. **Un encodeur ViT-Tiny** $f(\cdot)$ produisant une représentation globale du token [CLS].
2. **Un projecteur MLP** $g(\cdot)$ à deux couches, qui mappe la représentation dans un espace latent de dimension 128.
3. **La perte contrastive NT-Xent**, appliquée sur les paires positives (z_i, z_j) issues des deux vues d'une même image.

Cette architecture permet d'apprendre des représentations invariantes aux transformations visuelles, qui serviront ensuite de base au fine-tuning supervisé et au Test-Time Training.

3.2.2 Vision Transformer ViT-Tiny

Nous utilisons un **Vision Transformer ViT-Tiny** :

- patch size 4×4 ;
- dimension d'embedding 192 ;
- 12 blocs Transformer ;
- LayerNorm avant chaque bloc.

3.2.3 Augmentations de données

3.2.4 Perte NT-Xent

3.3 Fine-tuning supervisé

Après le pré-entraînement auto-supervisé, le modèle est affiné sur CIFAR-10 à l'aide d'un entraînement supervisé classique. Cette étape vise à spécialiser les représentations apprises par SimCLR pour la tâche de classification, tout en conservant une tête auxiliaire indispensable au Test-Time Training.

3.3.1 Tête de classification

La tête principale est une couche linéaire appliquée au token [CLS] du ViT-Tiny. Elle prédit les 10 classes de CIFAR-10 à l'aide d'une softmax. L'ensemble du backbone est dégelé, permettant au modèle d'ajuster finement ses représentations à la tâche supervisée.

Les hyperparamètres utilisés pour cette étape sont :

- **epochs : 30** — durée d'entraînement supervisé ;
- **learning_rate : $5 \cdot 10^{-4}$** — taux d'apprentissage initial ;
- **weight_decay : 0.05** — régularisation L2 ;
- **warmup_epochs : 3** — montée progressive du learning rate ;
- **use_amp : true** — entraînement en précision mixte ;
- **early_stopping_patience : 10** — arrêt anticipé basé sur la validation.

3.3.2 Tête auxiliaire de rotation

Une seconde tête est ajoutée pour prédire l'angle de rotation appliqué à l'image parmi quatre classes :

$$\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}.$$

Cette tête est entraînée conjointement avec la tête de classification. Elle n’est pas utilisée pour la prédiction finale, mais elle est essentielle pour le Test-Time Training : elle fournit un signal auto-supervisé permettant d’adapter le modèle au moment du test, même en l’absence de labels.

3.3.3 Fonction de perte

La perte totale combine deux termes :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cls}} + \lambda_{\text{rot}} \mathcal{L}_{\text{rot}},$$

où :

- $\mathcal{L}_{\text{cls}}$ est la cross-entropy de classification, avec un **label smoothing de 0.1** pour améliorer la stabilité ;
- $\mathcal{L}_{\text{rot}}$ est la cross-entropy de rotation ;
- λ_{rot} est un poids fixé à 1 dans notre configuration.

Cette combinaison permet au modèle d’apprendre simultanément une représentation discriminante pour la classification et une représentation sensible aux transformations géométriques, exploitable pour l’adaptation au test.

3.4 Test-Time Training

Le Test-Time Training (TTT) consiste à adapter le modèle au moment du test en exploitant une tâche auto-supervisée disponible même en absence de labels. Dans notre cas, la tâche auxiliaire est la prédiction de rotation, introduite par Sun et al. (2020). Cette approche permet au modèle de s’ajuster dynamiquement aux corruptions présentes dans les données de test, améliorant ainsi sa robustesse.

Notre configuration TTT est définie dans le fichier `configs/default.yaml` et repose sur les hyperparamètres suivants :

- **method** : “**sun2020**” — implémentation fidèle du TTT original ;
- **steps** : 1 — un pas de gradient par adaptation ;
- **learning_rate** : 10^{-3} — taux d’apprentissage utilisé uniquement au test ;
- **adapt_scope** : “**encoder_plus_head**” — adaptation conjointe de l’encodeur ViT et de la tête rotation ;
- **lambda_rot** : 1.0 — poids de la perte rotation ;
- **rotation_mode** : “**rand**” — rotation aléatoire parmi les quatre angles ;
- **eval_mode** : “**both**” — exécution des deux régimes : per-batch et per-image ;

- **per_image_aug_k : 128** — nombre de copies tournées par image ;
- **per_image_subsample : 1000** — sous-échantillon stratifié pour limiter le coût.

3.4.1 Principe du TTT

Pendant l’inférence, les labels de classification ne sont pas disponibles. Cependant, la tâche auxiliaire de rotation reste exploitable : pour chaque image de test, on peut générer une version tournée et calculer la perte rotation correspondante.

Le TTT suit donc les étapes suivantes :

1. générer une ou plusieurs rotations de l’image de test ;
2. calculer la perte de rotation $\mathcal{L}_{rot}$;
3. effectuer un ou plusieurs pas de gradient sur les paramètres du modèle ;
4. produire la prédiction finale via la tête de classification.

Cette adaptation locale permet au modèle de corriger ses représentations en fonction de la corruption rencontrée.

3.4.2 Adaptation per-batch

Dans le mode **per-batch**, un pas de gradient est effectué pour chaque batch de test. Ce régime est stable car :

- il exploite plusieurs images simultanément ;
- il amortit le bruit du gradient ;
- il s’adapte progressivement aux corruptions ;
- il est compatible avec les propriétés statistiques des représentations ViT.

Dans nos expériences, ce mode est celui qui fournit les meilleurs résultats, avec un gain moyen de +0.39 pp sur CIFAR-10-C.

3.4.3 Adaptation per-image

Le mode **per-image** applique l’adaptation individuellement à chaque image. Pour stabiliser le gradient, chaque image est dupliquée en **128 copies tournées** (paramètre **per_image_aug_k**). Cependant, ce mode est coûteux et instable, en particulier avec les architectures ViT utilisant LayerNorm :

- le signal auto-supervisé est faible (une seule image) ;
- LayerNorm ne bénéficie pas des statistiques de batch ;
- l’encodeur peut dériver rapidement ;
- les performances se dégradent sur 46 cellules sur 70 dans nos résultats.

Pour limiter le coût, nous appliquons ce mode uniquement sur un **sous-échantillon stratifié de 1 000 images**.

3.4.4 Scope d’adaptation

Le paramètre `adapt_scope` contrôle les parties du modèle mises à jour pendant l’adaptation. Nous utilisons :

- **encoder_plus_head** : adaptation conjointe du backbone ViT-Tiny et de la tête rotation.

La tête de classification reste figée afin de préserver la fonction de décision. Ce choix suit la formulation originale de Sun et al. et permet d’éviter une dérive excessive des logits de classification.

Ce scope est crucial :

- un scope trop large entraîne une sur-adaptation (observée en per-image) ;
- un scope trop restreint limite l’efficacité du TTT.

Chapitre 4

Étude expérimentale

4.1 Datasets

4.1.1 CIFAR-10

CIFAR-10 est un jeu de données composé de 60 000 images couleur de taille 32×32 , réparties en 10 classes équilibrées. Il est divisé en :

- 50 000 images d’entraînement,
- 10 000 images de test.

Pour le pré-entraînement SimCLR, nous utilisons uniquement les images d’entraînement, sans labels. Pour les stages supervisés, un split interne est réalisé afin de constituer un ensemble de validation.

4.1.2 CIFAR-10-C

CIFAR-10-C est un benchmark conçu pour évaluer la robustesse des modèles face aux corruptions. Il contient :

- 15 types de corruptions (bruit, flou, météo, compression, etc.),
- 5 niveaux de sévérité,
- 10 000 images par corruption et par sévérité.

Ce dataset est utilisé exclusivement pour le Stage C (TTT). Le pipeline télécharge automatiquement l’archive depuis Zenodo, la met en cache sur Google Drive et l’extrait localement.

4.1.3 GPU et environnement

Toutes les expériences ont été réalisées sur un GPU **NVIDIA L4** dans un environnement Python basé sur **PyTorch 2.x** et **CUDA 12**. L’ensemble du pipeline — pré-entraînement SimCLR, fine-tuning supervisé, Test-Time Training et évaluation CIFAR-

10-C — est orchestré via un fichier de configuration YAML, garantissant la reproductibilité des résultats. Toutes les expériences utilisent la seed **42**.

4.2 Setup expérimental

L'évaluation complète (Stage C) consiste en :

- **14 corruptions** de CIFAR-10-C¹ ;
- **5 niveaux de sévérité** par corruption ;
- soit un total de **70 cellules** d'évaluation.

Pour chaque cellule (corruption, sévérité), nous calculons :

- **baseline** : 10 000 images, sans adaptation ;
- **TTT per-batch** : 10 000 images, un pas de gradient par batch ;
- **TTT per-image** : sous-échantillon stratifié de 1 000 images, un pas par image.

Le mode per-image est limité à 1 000 images en raison de son coût computationnel : chaque image est dupliquée en un batch artificiel de 128 rotations, ce qui représente environ **20 heures de calcul** sur GPU L4 pour l'ensemble des 70 cellules.

Afin d'éviter toute fuite d'adaptation entre cellules, les poids du modèle sont **réinitialisés** avant chaque nouvelle combinaison (corruption, sévérité).

4.2.1 Hyperparamètres

Les hyperparamètres du fine-tuning supervisé (Stage B.2) sont présentés dans le Tableau 4.1. Ils correspondent à la configuration utilisée pour entraîner le modèle final avant l'adaptation TTT.

Paramètre	Valeur
Optimiseur	AdamW
Learning rate	$5 \cdot 10^{-4}$
Batch size	128
Époques	100
Scheduler	Cosine decay
Warmup	5 époques
Label smoothing	0.1
AMP	Oui

TABLE 4.1 – Hyperparamètres du fine-tuning supervisé.

Pour mémoire, le pré-entraînement SimCLR (Stage A) utilise une configuration plus lourde :

- batch size : 1024 ;

1. 14 sur les 15 corruptions standard de Hendrycks & Dietterich [7]. Le détail est fourni en Annexe A.

- 200 époques ;
- AdamW, lr $3 \cdot 10^{-3}$;
- warmup 10 époques ;
- scheduler cosine ;
- AMP activé.

4.2.2 Protocoles d'évaluation

L'évaluation robuste est réalisée sur **CIFAR-10-C**, qui applique quatorze corruptions visuelles, chacune déclinée en cinq niveaux de sévérité. Ces corruptions simulent différents types de *distribution shift* rencontrés en conditions réelles.

Dans ce rapport, nous mettons en avant quatre familles représentatives :

- **Gaussian noise** : bruit additif aléatoire, affectant fortement les textures ;
- **Blur** : flou gaussien ou flou de mouvement, dégradant les contours ;
- **Rotation** : transformations géométriques perturbant la structure globale ;
- **Brightness shift** : variations d'illumination, simulant des conditions d'éclairage difficiles.

Chaque corruption est évaluée selon trois modes :

- **baseline** : aucune adaptation ;
- **TTT per-batch** : un pas de gradient par batch de test ;
- **TTT per-image** : un pas de gradient par image (batch artificiel de 128 rotations).

Pour garantir l'indépendance des résultats, les poids du modèle sont réinitialisés avant chaque cellule d'évaluation.

4.3 Résultats sans TTT

Nous commençons par évaluer les performances du modèle sans aucune adaptation au moment du test. Le Tableau 4.2 présente les résultats sur CIFAR-10 clean.

Configuration	Top-1 Accuracy
Fine-tuning (val, meilleure époque)	0.9008
Baseline CIFAR-10 clean (sans TTT)	0.8959

TABLE 4.2 – Performances sur CIFAR-10 clean sans adaptation.

Ces résultats montrent que le modèle fine-tuné atteint une performance élevée sur les données propres, et que l'absence d'adaptation ne provoque pas de dégradation notable. Ce *sanity check* confirme que la tête auxiliaire de rotation n'affecte pas négativement la classification.

Sur CIFAR-10-C, l’accuracy moyenne (toutes corruptions et sévérités confondues) est de :

$$\text{baseline} = \mathbf{0.7883}.$$

Comme attendu, la performance décroît avec la sévérité des corruptions.

4.4 Résultat avec TTT

Nous évaluons deux régimes d’adaptation au test :

- **TTT per-batch** : un pas de gradient par batch de test ;
- **TTT per-image** : un pas de gradient par image (batch artificiel de 128 rotations).

CIFAR-10-C. Le Tableau 4.3 présente l’accuracy moyenne par sévérité.

Sévérité	baseline	per-batch	per-image (1k)	Δ batch	Δ image
1	0.8673	0.8679	0.8666	+0.0007	-0.0006
2	0.8392	0.8409	0.8395	+0.0017	+0.0003
3	0.8033	0.8059	0.8033	+0.0025	0.0000
4	0.7516	0.7575	0.7534	+0.0059	+0.0018
5	0.6803	0.6890	0.6830	+0.0087	+0.0027
moyenne	0.7883	0.7922	0.7892	+0.0039	+0.0009

TABLE 4.3 – Accuracy moyenne par sévérité sur CIFAR-10-C.

Le mode **per-batch** améliore systématiquement les performances, avec un gain croissant selon la sévérité. Le mode **per-image** est instable : il améliore légèrement aux sévérités faibles mais régresse sur le sous-ensemble apparié.

Mode	wins	losses	ties
TTT per-batch	61	6	3
TTT per-image (1k)	16	46	8

TABLE 4.4 – Nombre de cellules améliorées / dégradées sur CIFAR-10-C (70 cellules).

Wins / losses. Le mode per-batch suit la signature de Sun (2020). Le mode per-image dégrade dans 46 cellules sur 70.

Top gains per-batch. Les plus fortes améliorations concernent les corruptions lourdes (fog, pixelate, noise), avec des gains jusqu’à **+2.19 pp**.

4.4.1 Comparaison globale

La comparaison synthétique des méthodes est présentée dans le Tableau 4.5.

Méthode	Accuracy moyenne CIFAR-10-C
Baseline	0.7883
SSL + Fine-tuning	0.8959 (clean)
SSL + Fine-tuning + TTT per-batch	0.7922
SSL + Fine-tuning + TTT per-image	0.7892

TABLE 4.5 – Comparaison des performances globales.

Le TTT per-batch apporte un gain clair et régulier. Le TTT per-image reste instable dans notre configuration ViT/LayerNorm.

4.4.2 Effet des hyperparamètres

Les ablations montrent que :

- augmenter le nombre de steps améliore fortement le mode per-batch ;
- le mode per-image devient simultanément plus instable ;
- des learning rates trop faibles rendent l’adaptation inefficace.

Chapitre 5

Discussion

5.1 Pourquoi le per-batch fonctionne

Le mode per-batch suit fidèlement la signature observée dans les travaux de Sun et al. [1]. Il présente plusieurs caractéristiques positives :

- un gain moyen de **+0.39 pp** sur l’ensemble des corruptions ;
- une amélioration dans **61 cellules sur 70**, avec seulement 6 régressions mineures ;
- une absence totale de dégradation sur les données propres ;
- un gain croissant avec la sévérité, atteignant **+0.87 pp** à sévérité 5.

Ces résultats confirment que la tâche auxiliaire de rotation fournit un signal d’adaptation pertinent lorsque les corruptions sont suffisamment fortes.

5.2 Pourquoi le per-image échoue

À l’inverse, le mode per-image se révèle instable dans notre configuration ViT/LayerNorm :

- il dégrade **46 cellules sur 70** sur le sous-ensemble apparié ;
- son gain moyen est négatif (**-0.48 pp**) ;
- il est sensible au bruit d’optimisation, car chaque mise à jour repose sur un seul exemple.

Trois facteurs expliquent cette instabilité :

1. **ViT utilise LayerNorm** : contrairement aux modèles CNN avec BatchNorm, il ne bénéficie pas de la régularisation implicite des statistiques de batch.
2. **Le signal SSL est faible** : 128 copies tournées d’une seule image ne fournissent qu’un gradient peu informatif.
3. **Le scope d’adaptation est large** : adapter l’encodeur complet + la tête rotation peut provoquer une dérive rapide des représentations.

5.3 Limites de l'étude

Plusieurs limites doivent être soulignées :

- **Coût computationnel.** Le mode per-image nécessite environ **20 heures** de calcul sur GPU L4 pour 1 000 images, ce qui le rend difficilement applicable à grande échelle.
- **Sensibilité aux hyperparamètres.** Les performances varient fortement selon le learning rate, le nombre de pas d'adaptation et le scope des paramètres mis à jour. Une adaptation trop agressive entraîne une sur-adaptation.
- **Instabilité structurelle.** Les architectures ViT basées sur LayerNorm sont plus sensibles aux mises à jour locales que les CNN avec BatchNorm, ce qui explique l'échec du mode per-image.
- **Ablations incomplètes.** Seuls `steps` et `lr` ont été explorés. Les hyperparamètres `adapt_scope`, `lambda_rot` et `rotation_mode` restent à étudier.
- **Absence de comparaison avec d'autres méthodes d'adaptation.** Des approches comme Tent, BN-adaptation ou ActMAD pourraient fournir des points de comparaison utiles.

Chapitre 6

Conclusion et perspectives

6.1 Conclusion

Ce travail avait pour objectif d’étudier l’efficacité du *Test-Time Training* (TTT) auto-supervisé pour améliorer la robustesse d’un modèle de classification d’images face aux corruptions de distribution, en particulier dans le cadre d’un backbone Vision Transformer (ViT-Tiny) pré-entraîné avec SimCLR.

Nous avons d’abord mis en place un pipeline complet et reproductible comprenant : (i) un pré-entraînement auto-supervisé contrastif, (ii) un fine-tuning supervisé avec une tête auxiliaire de rotation, (iii) une évaluation sur CIFAR-10-C, (iv) deux régimes d’adaptation au test : *per-batch* et *per-image*.

Les résultats montrent que :

- le TTT **per-batch** améliore systématiquement la robustesse du modèle, avec un gain moyen de +0.39 pp et jusqu’à +0.87 pp aux sévérités les plus élevées ;
- cette amélioration est cohérente avec la signature observée dans les travaux de Sun et al. (2020) ;
- le TTT **per-image** est instable dans notre configuration ViT/LN, dégradant les performances dans 46 cellules sur 70 ;
- l’ablation **steps=5** confirme l’hypothèse de sur-adaptation : le per-batch s’améliore fortement (+1.50 pp), tandis que le per-image s’effondre (-1.68 pp) ;
- les gains se concentrent sur les corruptions lourdes (fog, noise, pixelate), où la tâche de rotation fournit un signal d’adaptation pertinent.

Ces observations montrent que le TTT reste une approche efficace pour améliorer la robustesse des modèles, mais que son comportement dépend fortement de l’architecture sous-jacente. En particulier, les Vision Transformers basés sur LayerNorm réagissent différemment des CNN à BatchNorm, ce qui remet en question l’applicabilité directe des méthodes conçues pour les architectures convolutionnelles.

6.2 Perspectives

Plusieurs pistes d'amélioration et d'exploration future émergent de ce travail :

- **Étendre les ablations.** Explorer d'autres hyperparamètres du TTT : `adapt_scope`, `lambda_rot`, `rotation_mode`, nombre de pas d'adaptation, ou encore des stratégies d'optimisation plus stables.
- **Tester d'autres tâches auto-supervisées.** La rotation est simple mais peut être sous-optimale pour les ViT. Des tâches comme MAE, DINO, iBOT ou des prétextes géométriques plus riches pourraient fournir un signal plus robuste.
- **Comparer avec d'autres méthodes d'adaptation au test.** Intégrer Tent, TTT++, EATA, SAR ou ActMAD permettrait de situer précisément les performances du TTT dans un paysage plus large.
- **Analyser l'effet de LayerNorm.** Étudier pourquoi les mises à jour locales déstabilisent les ViT pourrait mener à des variantes de TTT mieux adaptées aux architectures Transformer.
- **Évaluer sur des datasets plus complexes.** CIFAR-10-C reste un benchmark simple. Tester sur Tiny-ImageNet-C ou ImageNet-C permettrait de valider la scalabilité du pipeline.
- **Réduire le coût du per-image.** Le mode per-image est coûteux ($\approx 20h$ pour 10k images). Des stratégies d'approximation ou d'adaptation partielle pourraient le rendre exploitable.
- **Stabiliser l'adaptation.** Introduire des mécanismes de régularisation (EMA, poids gelés partiellement, clipping du gradient) pourrait limiter la sur-adaptation observée.

En résumé, ce travail montre que le TTT per-batch constitue une approche simple et efficace pour améliorer la robustesse des modèles SSL basés sur ViT, tandis que le per-image nécessite des adaptations méthodologiques pour être pleinement exploitable. Ce projet ouvre la voie à de nombreuses explorations futures, tant sur le plan algorithmique que sur celui de la compréhension des mécanismes internes des Vision Transformers.

Bibliographie

- [1] Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, M. Hardt, *Test-time training with self-supervision for generalization under distribution shifts*, ICML, 2020. [arXiv:1909.13231](#).
- [2] Y. Liu, P. Kothari, B. Van Delft, B. Bellot-Gurlet, T. Mordan, A. Alahi, *TTT++ : When does self-supervised test-time training fail or thrive ?*, NeurIPS, 2021.
- [3] J. Han, J. Park, D. Han, W. Hwang, *When Test-Time Adaptation Meets Self-Supervised Models*, [arXiv:2506.23529](#), 2025.
- [4] M. J. Mirza, P. Jané Soneira, W. Lin, M. Kozinski, H. Possegger, H. Bischof, *Act-MAD : Activation matching to align distributions for test-time training*, CVPR, 2023.
- [5] T. Chen, S. Kornblith, M. Norouzi, G. Hinton, *A Simple Framework for Contrastive Learning of Visual Representations (SimCLR)*, ICML, 2020. [arXiv:2002.05709](#).
- [6] A. Dosovitskiy *et al.*, *An Image is Worth 16x16 Words : Transformers for Image Recognition at Scale (ViT)*, ICLR, 2021. [arXiv:2010.11929](#).
- [7] D. Hendrycks, T. Dietterich, *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations (CIFAR-10-C)*, ICLR, 2019. [arXiv:1903.12261](#).
- [8] S. Gidaris, P. Singh, N. Komodakis, *Unsupervised Representation Learning by Predicting Image Rotations*, ICLR, 2018. [arXiv:1803.07728](#).
- [9] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images (CIFAR-10/100)*, Technical Report, University of Toronto, 2009.